

RDAH-Net

Method 5: RDAH-Net

1. Objective and role of RDAH-Net in DepthWizard2

RDAH-Net was investigated as a learned alternative to the simpler metric-calibration and correction methods developed earlier in DepthWizard2.

The intended role was to take:

RGB image + frozen DAv2 relative-depth prior $\rightarrow$ metric height / AGL prediction

and to learn a spatially varying transformation from the relative depth representation into metric height, rather than relying only on a global calibration relationship.

The investigation was conducted primarily on the DFC2019 benchmark, using the existing 50-tile spatial benchmark defined for DepthWizard2.

The RDAH experiment was deliberately kept separate from the earlier methods. The pretrained RDAH architecture and checkpoint were used first as supplied, then adapted to the DepthWizard2/DFC2019 data through controlled fine-tuning.

2. RDAH-Net source and repository inspection

The repository was already present inside the project at:

```
external/RDAH-Net/  
├── HK-test.txt  
├── HK-train.txt  
├── LICENSE  
├── loaddata.py  
├── nyu_transform.py  
├── README.md  
├── requirements.txt  
├── Swiss-test.txt  
├── Swiss-train.txt  
├── Track1-test.txt  
├── Track1-train.txt  
├── test.py  
└── train.py
```

The repository contains no additional model subdirectories.

The exact pretrained checkpoint located and used for all experiments was:

external/RDAH-Net/104best_model.pth The checkpoint is approximately 62 MB.

Earlier assumptions about an alternate checkpoint location such as:

models/external/rdah_net/track1_best_model.pth were corrected. That path was not the checkpoint actually used.

No further checkpoint archaeology was pursued after the correct **104best_model.pth** was established.

3. Checkpoint inspection

The checkpoint was loaded directly with:

`torch.load(..., map_location="cpu")` Its top-level keys were:

`['epoch', 'model_state_dict', 'optimizer_state_dict', 'loss']` The stored checkpoint metadata showed:

`epoch = 48 loss = 1.205092347064421` The `model_state_dict` contained 506 tensors.

The model architecture was identified as:

HeightPredTransformer with the following major components:

dual MobileViT_S_Light encoders

one for depth input

one for RGB input

CBAM modules at three scales

three bidirectional LightCrossAttention blocks

depth $\rightarrow$ image cross-attention

image $\rightarrow$ depth cross-attention

positional encoding

global LightTransformerBlock

skip projections

PixelShuffle decoder

The native architecture was designed for 1024×1024 inputs.

Its approximate spatial progression was:

```
1024
↓ stem
512
↓ stage 1
256
↓ stage 2
128
↓ stage 3
64
↓ decoder / PixelShuffle
1024
The architectural inspection established that RDAH was not simply a scalar depth-
calibration model; it was a multimodal RGB/depth fusion network intended to
produce a dense metric height prediction.
```

4. Original RDAH data interface

The original RDAH loader was inspected before adaptation.

The source pipeline reads:

RGB relative depth target height The original training/testing code passes:

`prediction = model(depth_input, image_input)` and evaluates the prediction against the height target using the model's loss/evaluation machinery.

The original loss was a masked SmoothL1-style loss. The relevant source implementation was inspected before reproducing the training setup.

The original RGB preprocessing used ImageNet normalization.

The original relative-depth path did not use ImageNet normalization; the relative- depth image was passed directly through tensor conversion.

This distinction was preserved in the DepthWizard2 adaptation.

The RDAH paper/source formulation was also consistent with treating relative depth as a dimensionless prior that is subsequently fused with RGB information.

5. Adaptation of RDAH to DepthWizard2

The DepthWizard2 experiment was designed around the already established DFC2019 benchmark. The inputs were:

DFC2019 RGB + Depth Anything V2 relative-depth prediction The target was:

DFC2019 AGL The DAV2 cache used for the experiment was:

`data/dfc2019/experiments/dav2_baseline/depth/<tile>_depth.npy` These DAV2 depth arrays were stored as float32 values normalized to:

`[0, 1]` The DFC2019 benchmark was the same 50-tile benchmark already used elsewhere in the project.

The manifest was:

`data/dfc2019/experiments/dav2_baseline/manifest.csv` The adaptation deliberately retained:

the official RDAH architecture

the pretrained checkpoint

the RDAH-style RGB preprocessing

direct DAV2 relative-depth input

the official-style masked SmoothL1 loss

batch size 1

no augmentation

MPS execution on the Mac

learning rate $1e-5$

The purpose was to isolate the effect of domain adaptation/fine-tuning rather than introduce architectural or preprocessing changes simultaneously. 6. Initial standalone RDAH probe Before fine-tuning, the pretrained checkpoint was tested directly against DepthWizard2 inputs.

A standalone probe script was created:

`/mnt/data/run_rdah_probe.py` and subsequently used as:

`scripts/run_rdah_probe.py` The probe:

- 1. imported the official RDAH HeightPredTransformer;**
- 2. loaded external/RDAH-Net/104best_model.pth;**
- 3. loaded a DFC2019 RGB tile;**
- 4. loaded the corresponding DAv2 .npy;**
- 5. applied the appropriate RGB preprocessing;**
- 6. supplied the DAv2 relative depth directly;**
- 7. recorded output statistics.**

Four representative tiles were tested.

6.1 Probe: JAX_004_006

DAv2 statistics:

min = 0 max = 1 mean = 0.299208 std = 0.168367 RDAH prediction:

min = -0.334840 max = 2.246881 mean = 0.081055 std = 0.014324

6.2 Probe: JAX_149_006

DAv2:

mean = 0.265984 std = 0.226479 RDAH prediction:

min = -0.330024 max = 1.761953 mean = 0.080858 std = 0.011393

6.3 Probe: JAX_264_013

DAv2:

mean = 0.439860 std = 0.221178 RDAH prediction:

min = -0.340304 max = 1.886261 mean = 0.081000 std = 0.012731

6.4 Probe: OMA_248_029

DAv2:

mean = 0.339765 std = 0.192927 RDAH prediction:

min = -0.323679 max = 1.766758 mean = 0.081013 std = 0.013057

7. Initial probe inference

Across different DFC2019 scenes, the pretrained RDAH output had an extremely stable mean of approximately:

0.081 and a very small standard deviation, approximately:

0.011–0.014 despite substantial variation in the DAv2 inputs.

This established that the pretrained RDAH checkpoint, without adaptation, was not operating as a useful metric predictor in the DepthWizard2/DFC2019 domain.

The investigation therefore moved to determine whether the problem was merely an input-scale or depth-orientation mismatch before proceeding to fine-tuning.

8. RDAH input-scale sweep

A dedicated scale-sweep experiment was created:

`scripts/run_rdah_scale_sweep.py` The experiment was run on:

JAX_149_006 The RDAH model was evaluated with the DAv2 depth input multiplied by different factors.

The results were:

- baseline : mean 0.0809, std 0.0114
- $\times 0.001$: mean 0.0809, std 0.0115
- $\times 0.01$: mean 0.0809, std 0.0115
- $\times 0.1$: mean 0.0809, std 0.0115

- $\times 1$: mean 0.0809, std 0.0114
- $\times 10$: mean 0.0810, std 0.0111
- $\times 100$: mean 0.3472, std 0.8175
- $\times 1000$: mean 8.8923, std 9.8402
- invert : mean 0.0809, std 0.0114
- The experiment therefore showed that ordinary changes in depth scale or inversion
- did not solve the near-constant-output behavior.

The extreme $\times 100$ and $\times 1000$ cases changed the output substantially, but these were artificial and were not treated as valid model inputs.

The input-scale sweep was therefore closed.

No arbitrary $\times 100$ or $\times 1000$ transformation was adopted.

No further zero-shot scale probing was pursued.

The conclusion from this stage was that input scale/orientation was not the root cause of the zero-shot failure.

That justified moving to domain fine-tuning rather than continuing zero-shot probing.

9. Design of the spatial fine-tuning experiment

A dedicated fine-tuning script was created:

scripts/train_rdah_spatial_cv.py The purpose was to test whether RDAH could adapt to the DFC2019 metric-height domain while preserving spatial holdout.

The experiment used the same 50-tile benchmark manifest:

data/dfc2019/experiments/dav2_baseline/manifest.csv The model was initialized from:

external/RDAH-Net/104best_model.pth The 50 benchmark tiles were divided into four spatial folds.

- The resulting fold sizes were:
 - fold 0: 17 tiles
 - fold 1: 8 tiles
 - fold 2: 7 tiles
 - fold 3: 18 tiles
 - Thus:
- ```
fold 0 → train 33 / test 17
fold 1 → train 42 / test 8
fold 2 → train 43 / test 7
fold 3 → train 32 / test 18
```
- The split was performed at the tile level, keeping complete  $1024 \times 1024$  tiles intact.

This avoided pixel-level train/test leakage while respecting the native RDAH input design.

The training configuration remained deliberately conservative:

batch size = 1 learning rate =  $1e-5$  epochs = initially 1 augmentation = none loss = masked SmoothL1 device = MPS

## 10. First fine-tuning attempt and discovery of the target-dtype bug

The first Fold 0 experiment used one epoch.

It initially produced apparently catastrophic evaluation RMSE values such as:

```
JAX_031_006 → RMSE 263.9863
JAX_118_012 → RMSE 181.0760
JAX_204_005 → RMSE 247.9172
OMA_198_002 → RMSE 520.5465
```

The aggregate evaluation appeared to be:

| Metric   | Value    |
|----------|----------|
| MAE      | 4.5732   |
| RMSE     | 168.9284 |
| Pearson  | -0.0008  |
| Spearman | -0.0000  |

This was initially interpreted as a potential numerical failure, so the fine-tuned model was separately diagnosed against the original pretrained checkpoint.

The diagnostic used raw float32 AGL, rather than the training adapter's target representation.

That diagnostic showed that the apparently catastrophic RMSE was not actually caused by unstable model predictions.

The underlying problem was in the target-loading code.

## 11. Critical target-dtype bug

The first training adapter contained:

```
target_uint16 = np.where(valid, arr, 0).astype(np.uint16) target = torch.from_numpy(target_uint16.astype(np.float32))
```

 DFC2019 AGL contains valid negative height values.

The observed target minimum was approximately:

-2.132 m Casting those signed values to uint16 caused them to wrap around to very large positive values.

This created invalid target heights near the upper end of the unsigned 16-bit range and therefore corrupted the RMSE calculation and training target.

The corrected code replaced the unsigned conversion with float32 preservation:

```
target = torch.from_numpy(np.where(valid, arr, 0.0).astype(np.float32))
```

 The target mask continued to identify valid pixels.

The corrected script was rewritten and supplied as:

**/mnt/data/train\_rdah\_spatial\_cv.py** The corrupted Fold 0 output directory was removed and Fold 0 was rerun.

This correction was essential because otherwise subsequent RDAH training results would have been invalid.

## 12. Diagnostic comparison after correcting the target representation

Before rerunning the training, a diagnostic compared:

pretrained RDAH vs. 1-epoch fine-tuned RDAH on the same Fold 0 held-out tiles using the correct raw float32 AGL targets.

The pretrained model produced:

prediction range  $\approx$  [-0.3505, 2.4194] prediction mean  $\approx$  0.0810 prediction std  $\approx$  0.0132 Ground truth:

range  $\approx$  [-2.1320, 136.7200] mean  $\approx$  4.3818 std  $\approx$  7.7907 Aggregate pretrained metrics:

| Metric   | Value    |
|----------|----------|
| MAE      | 4.3657 m |
| RMSE     | 8.8988 m |
| Pearson  | 0.0092   |
| Spearman | -0.0589  |

The one-epoch fine-tuned model:

prediction range  $\approx$  [-1.5115, 0.5880] mean  $\approx$  0.0270 std  $\approx$  0.0787 Aggregate metrics: MAE = 4.3871 m RMSE = 8.9277 m Pearson = -0.0322 Spearman = 0.0089 This established that the one-epoch fine-tuning stage had not yet produced meaningful spatial correlation.

The corrected one-epoch Fold 0 run subsequently confirmed:

| Metric   | Value    |
|----------|----------|
| MAE      | 4.3845 m |
| RMSE     | 8.9258 m |
| Pearson  | -0.0315  |
| Spearman | 0.0100   |

The model was numerically stable, but not yet useful.

At that point, one epoch was not sufficient evidence to reject the approach.

## 13. Five-epoch Fold 0 experiment

Fold 0 was therefore extended to five epochs without changing any architecture, loss, learning rate, preprocessing, or data split.

Fold 0 contained:

train = 33 test = 17 The complete held-out progression was:

| Epoch | MAE    | RMSE   | Pearson | Spearman |
|-------|--------|--------|---------|----------|
| 1     | 4.3845 | 8.9258 | -0.0315 | 0.01     |
| 2     | 4.4079 | 8.9148 | 0.1567  | 0.0838   |
| 3     | 3.8711 | 8.0348 | 0.6505  | 0.537    |

| Epoch | MAE    | RMSE   | Pearson | Spearman |
|-------|--------|--------|---------|----------|
| 4     | 3.6317 | 7.6734 | 0.6182  | 0.5775   |
| 5     | 3.6205 | 7.6663 | 0.6066  | 0.5732   |

The fold showed a large transition between epochs 2 and 3.

By epoch 3 the model had developed a substantial positive relationship with the held-out height field. Absolute errors continued improving into epoch 4–5, while correlation began to level off and slightly decline.

## 14. Fold 1 five-epoch experiment

Fold 1 contained:

train = 42 test = 8 The complete held-out progression was:

| Epoch | MAE    | RMSE   | Pearson | Spearman |
|-------|--------|--------|---------|----------|
| 1     | 2.2191 | 4.9123 | 0.0797  | 0.1234   |
| 2     | 2.1511 | 4.7527 | 0.4309  | 0.2994   |
| 3     | 2.1493 | 4.5708 | 0.2973  | 0.3296   |
| 4     | 2.1842 | 4.5893 | 0.2709  | 0.3235   |
| 5     | 2.207  | 4.6228 | 0.2535  | 0.3072   |

The fold showed clear learning between epochs 1 and 2.

Its best absolute performance occurred around epoch 3, after which MAE and correlation both began to deteriorate.

The fold therefore provided the clearest evidence of mild later-epoch overfitting.

The complete Fold 1 log is recorded in the uploaded experiment output.

## 15. Fold 2 five-epoch experiment

Fold 2 contained:

train = 43 test = 7 The held-out progression was:

| Epoch | MAE    | RMSE   | Pearson | Spearman |
|-------|--------|--------|---------|----------|
| 1     | 3.6803 | 8.3948 | -0.03   | 0.0169   |
| 2     | 3.5109 | 8.0564 | 0.5147  | 0.3338   |
| 3     | 3.213  | 7.7259 | 0.3882  | 0.5126   |
| 4     | 3.2012 | 7.7305 | 0.3668  | 0.5086   |
| 5     | 3.1997 | 7.7226 | 0.3634  | 0.5026   |

Again, the model transitioned from approximately zero correlation at epoch 1 to meaningful positive correlation after adaptation.

MAE continued to improve through epoch 5, while Pearson and Spearman peaked earlier.

The complete Fold 2 progression is present in the experiment log.

## 16. Fold 3 five-epoch experiment

Fold 3 contained:

train = 32 test = 18 The held-out progression was:

| Epoch | MAE    | RMSE   | Pearson | Spearman |
|-------|--------|--------|---------|----------|
| 1     | 3.0269 | 6.9464 | 0.0159  | 0.0648   |
| 2     | 3.0625 | 6.9399 | 0.1935  | 0.0594   |
| 3     | 2.6131 | 6.1711 | 0.6513  | 0.5496   |
| 4     | 2.4319 | 5.9115 | 0.5735  | 0.5624   |
| 5     | 2.4279 | 5.9152 | 0.547   | 0.5544   |

Fold 3 showed one of the largest transitions.

Epoch 1 was effectively uncorrelated:

Pearson = 0.0159 Spearman = 0.0648 By epoch 3:

Pearson = 0.6513 Spearman = 0.5496 Epoch 4 produced the best RMSE and near-best MAE, while epoch 5 changed little.

The complete progression is recorded in the uploaded output. 17. Cross-fold pattern The four folds produced a remarkably consistent qualitative trajectory.

At epoch 1:

Fold 0 Pearson = -0.0315 Fold 1 Pearson = 0.0797 Fold 2 Pearson = -0.0300 Fold 3 Pearson = 0.0159 Thus all four folds started near zero correlation.

After adaptation:

Fold 0 → Pearson peak ≈ 0.6505  
Fold 1 → Pearson peak ≈ 0.4309  
Fold 2 → Pearson peak ≈ 0.5147  
Fold 3 → Pearson peak ≈ 0.6513  
Similarly, Spearman correlation moved from near-zero values toward positive spatial rank correlation across the folds.

The equal-weight average fold metrics were approximately:

| Epoch | MAE   | RMSE  | Pearson | Spearman |
|-------|-------|-------|---------|----------|
| 1     | 3.328 | 7.295 | 0.009   | 0.054    |
| 2     | 3.283 | 7.166 | 0.324   | 0.194    |
| 3     | 2.962 | 6.626 | 0.497   | 0.482    |
| 4     | 2.862 | 6.476 | 0.457   | 0.493    |
| 5     | 2.864 | 6.482 | 0.443   | 0.484    |

These are equal-weight averages of the four fold-level metrics, not pooled pixel-level metrics.

The fold sizes are uneven:

17, 8, 7, 18 so these averages were treated as diagnostic summaries rather than the final benchmark statistic. 18. What the four folds establish The main finding is not simply that five epochs produce lower MAE.

The more important finding is the repeatability of learning.

The same pretrained model began with almost no held-out correlation across all four folds.

After fine-tuning, every fold developed positive correlation and positive rank correlation.

The effect was particularly visible around epochs 3–4.

This indicates that the model is learning a spatially varying relationship between the DAV2 prior, RGB information, and DFC2019 AGL rather than merely preserving its original near-constant output.

The consistency across four spatial holdouts is important because the central failure mode tested in earlier DepthWizard2 methods was inability to generalize spatially.

RDAH did not collapse under the spatial holdout protocol.

## 19. Epoch behavior and checkpoint selection

The experiments do not support declaring a universal optimum at epoch 5.

Instead:

Fold 0 continued improving absolute error through epoch 5.

Fold 1 peaked around epoch 3 and then degraded.

Fold 2 continued to make small MAE improvements through epoch 5, while correlation declined after earlier peaks.

Fold 3 reached its main improvement by epoch 3–4 and then plateaued. Thus the general behavior was:

Epoch 1: little useful spatial structure Epoch 2: adaptation begins to appear Epoch 3: strong improvement and emergence of spatial correlation Epoch 4: best overall region for several folds Epoch 5: mostly plateauing, with some evidence of mild overfitting This means epoch selection cannot simply be based on the lowest test-fold MAE after inspecting all epochs.

A final benchmark should use a pre-specified validation/checkpoint-selection protocol, or explicitly label the present results as exploratory spatial-CV results.

## 20. Relation to the earlier DepthWizard2 methods

RDAH was investigated after the earlier correction approaches had already been evaluated.

The earlier methods established several important constraints.

Method 2 — sparse-anchor metric calibration

The final Method 2 baseline was a simple global calibration using a 20-anchor grid.

Its representative result was approximately:

| Metric   | Value  |
|----------|--------|
| MAE      | 3.13 m |
| RMSE     | 4.48 m |
| Pearson  | 0.526  |
| Spearman | 0.463  |

More complex sparse-anchor variants, including Huber, RANSAC and spatial residual correction, were evaluated, and the spatial corrections did not justify replacing the simple global calibration.

Method 3 — semantic-conditioned correction

Method 3 improved same-tile/in-sample performance:

baseline: MAE 3.0349 RMSE 4.2933 Pearson 0.5457 Spearman 0.4775 versus:

Method 3: MAE 2.8975 RMSE 4.1231 Pearson 0.5843 Spearman 0.4901 However, under spatial holdout:

baseline: MAE 3.3924 RMSE 4.5787 Pearson 0.5824 Spearman 0.5093 while:

Method 3: MAE 3.3948 RMSE 4.9065 Pearson 0.5542 Spearman 0.4856 Therefore Method 3 improved in-sample behavior but failed to demonstrate spatial generalization and was retained as an ablation rather than a production correction.

Method 4 — learned RGB spatial correction Method 4 produced:

baseline: MAE 3.8817 RMSE 5.3409 Pearson 0.5876 Spearman 0.5112 and:

Method 4: MAE 3.2663 RMSE 5.1736 Pearson 0.5469 Spearman 0.4852 Its MAE improvement was accompanied by consistent decreases in Pearson and Spearman across all four folds.

It therefore remained promising but unresolved.

RDAH was subsequently investigated as a more substantial learned RGB/depth fusion approach.

## 21. What was ruled out during the RDAH investigation

Several possibilities were explicitly tested or eliminated.

Incorrect checkpoint path

Corrected to:

**external/RDAH-Net/104best\_model.pth** and used consistently afterward.

Zero-shot incompatibility caused by ordinary depth scaling

Tested:  $\times 0.001$   $\times 0.01$   $\times 0.1$   $\times 1$   $\times 10$  with essentially unchanged outputs.

Also tested:

1 - depth with essentially unchanged outputs.

The experiment therefore did not support ordinary scale or inversion as the solution.

Arbitrary extreme scaling

$\times 100$  and  $\times 1000$  changed predictions dramatically but were artificial and unstable.

They were rejected and not used in subsequent training.

Target corruption

The initial catastrophic RMSE values were traced to the adapter's uint16 target conversion.

That bug was corrected before interpreting the fine-tuning results.

Zero-shot rejection of RDAH

The initial near-constant zero-shot output was not used to permanently reject the model.

The subsequent controlled multi-epoch fine-tuning experiment demonstrated that the pretrained network could adapt substantially to the DFC2019 domain.

## 22. Reproducibility state

The principal scripts produced during the investigation are:

**scripts/run\_rdah\_probe.py scripts/run\_rdah\_scale\_sweep.py scripts/train\_rdah\_spatial\_cv.py** The pretrained checkpoint is:

**external/RDAH-Net/104best\_model.pth** The benchmark manifest is:

**data/dfc2019/experiments/dav2\_baseline/manifest.csv** The RDAH experiment output directory is:

**data/dfc2019/experiments/rdah/** The spatial-CV result file reported by the training script is:

`data/dfc2019/experiments/rdah/ method_rdah_spatial_cv_results.json` The native DAV2 cache remains:

`data/dfc2019/experiments/dav2_baseline/depth/` The RDAH fine-tuning was executed on:

MPS with:

batch size = 1 learning rate = 1e-5 epochs = 5 for the final four-fold exploratory run No augmentation was introduced.

## 23. Current methodological limitation

Although the four folds constitute a spatial holdout experiment, the current logged runs inspected the held-out test performance at every epoch.

Therefore, selecting a checkpoint solely because its test metrics happened to be best after seeing the results would introduce model-selection leakage into the final reported benchmark.

The current four-fold results should therefore be treated as: exploratory spatial-CV evidence demonstrating that RDAH can learn and generalize a metric-height relationship

rather than as the final unbiased model-selection benchmark.

A final production claim requires either:

an independent validation mechanism for selecting the training epoch, followed by a locked spatial test, or

a clearly predeclared checkpoint-selection protocol.

A further limitation is that the fold sizes are uneven, so the equal-weight averages above are not the definitive pooled 50-tile pixel-level statistics.

## 24. Ultimate conclusion — RDAH-Net

RDAH-Net began the investigation as a non-functional zero-shot candidate in the DepthWizard2/DFC2019 domain.

The pretrained checkpoint produced nearly constant outputs around mean  $\approx 0.081$  across substantially different DAV2 scenes, with essentially no useful spatial correlation.

An input-scale/inversion investigation did not resolve this behavior.

The first fine-tuning attempt then exposed a critical target-representation bug: valid negative DFC2019 AGL values were being cast to uint16, producing wrapped target values and artificially enormous RMSE figures. The training adapter was corrected to preserve signed float32 AGL values.

After correction, one epoch still produced essentially zero held-out correlation, so the model was not prematurely rejected.

The decisive experiment was the five-epoch four-fold spatial fine-tuning study.

Across all four spatial folds, the same qualitative transition occurred:

```
near-zero correlation
↓
adaptation
↓
positive spatial correlation
↓
substantial metric-height prediction structure
```

The fold results independently demonstrated this behavior. Pearson correlation rose from approximately zero at epoch 1 to values as high as approximately 0.43–0.65 across the folds, while Spearman correlation also became consistently positive.

At the same time, MAE and RMSE generally decreased substantially after adaptation.

The most important conclusion is therefore:

```
RDAH-Net successfully adapted from its pretrained domain to the
DFC2019/DepthWizard2 domain and demonstrated spatial generalization of a
learned RGB + DAV2-relative-depth → metric-AGL relationship under tile-level
spatial holdout.
```

This makes RDAH materially different from Method 3, whose apparent improvement disappeared under spatial evaluation.

It also makes RDAH a stronger learned candidate than the initial zero-shot investigation suggested.

However, the present evidence does not yet establish RDAH as the final DepthWizard2 production model or establish it as numerically superior to Method 2 or Method 4. The current four-fold runs are exploratory with respect to checkpoint selection, and the final comparison still requires a common, pooled, pre-specified evaluation protocol.

Accordingly, the scientifically supported status of RDAH at this stage is:

RDAH-Net is a validated and promising learned metric-reconstruction candidate, with demonstrated spatially generalizable adaptation on the DFC2019 benchmark, but final model selection and final cross-method comparison remain pending.

That is the point at which the RDAH investigation currently stands.